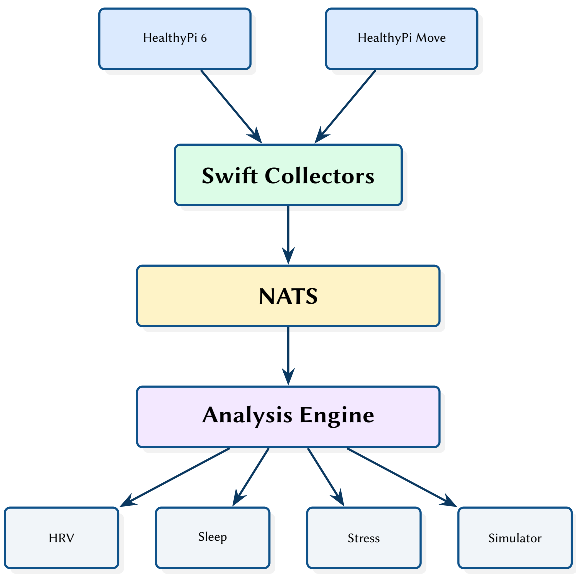
HealthyPi Agent: A Modular Health Monitoring Ecosystem

HRV Analysis, Sleep Scoring, and Agentic Intelligence for HealthyPi Hardware

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 11, 2026



Abstract. This report presents a modular health monitoring ecosystem that bridges ProtoCentral HealthyPi hardware (v6 and Move) with agentic intelligence. The Python analysis engine provides heart rate variability (HRV) analysis in both time and frequency domains, automated sleep stage scoring, EWMA baseline tracking with deviation detection, and stress/EDA analysis. A "Virtual Patient" simulator generates synthetic physiological data for development without hardware. Swift iOS/macOS collectors stream sensor data via NATS messaging, enabling real-time analysis and alerting within the multi-agent assistant ecosystem.

Contents

1 Introduction and Motivation

Consumer health devices produce valuable physiological data but typically lock analysis behind proprietary apps with limited algorithmic transparency. The HealthyPi platform from ProtoCentral [1] provides open-source hardware for Electrocardiogram (ECG), Photoplethysmography (PPG), SpO2, and temperature monitoring. This project extends the HealthyPi ecosystem with an agentic analysis engine that integrates with the multi-agent assistant for automated health insights.

2 System Architecture

The system has three layers: hardware collectors (Swift), a messaging backbone (NATS [2]), and the analysis engine (Python).

Table 1. Module organisation.

Module	Language	Role
`healthypi.analysis`	Python	HRV, sleep, stress analysis
`healthypi.models`	Python	Data models, Pydantic schemas
`healthypi.simulator`	Python	Virtual Patient generator
`healthypi.transforms`	Python	Signal processing pipeline
`healthypi.nats`	Python	NATS message handlers
`healthypi.menubar`	Python	macOS menu bar status app
Swift collectors	Swift	BLE → NATS bridge

3 HRV Analysis

The HRV module implements the Task Force standards [3] for both time-domain and frequency-domain analysis.

3.1 Time-Domain Metrics

Table 2. Time-domain HRV metrics.

Metric	Formula	Interpretation
Standard Deviation of NN Intervals (SDNN)	$\sqrt{\frac{1}{N}\sum(RR_i - \overline{RR})^2}$	Overall variability
Root Mean Square of Successive Differences (RMSSD)	$\sqrt{\frac{1}{N-1}\sum(RR_{i+1} - RR_i)^2}$	Parasympathetic activity
pNN50	$\frac{\text{count}(\lvert\Delta RR_i\rvert > 50\text{ms})}{N-1}$	Vagal tone

3.2 Frequency-Domain Analysis

The frequency-domain module computes power spectral density via Welch's method and extracts standard frequency bands:

- **VLF** (0.003–0.04 Hz) — thermoregulation, hormonal
- **LF** (0.04–0.15 Hz) — sympathetic + parasympathetic
- **HF** (0.15–0.40 Hz) — parasympathetic (respiratory)

The LF/HF ratio provides a measure of sympathovagal balance.

> **Design Decision 1: Artifact Rejection**
>
> RR intervals outside physiological bounds (200–2000ms) or differing from neighbours by more than 20% are flagged as artifacts and excluded from analysis. Interpolation (cubic spline) fills gaps for frequency-domain computation, which requires evenly-sampled data.

4 Sleep Scoring

The sleep scoring module classifies 30-second epochs into wake, light (N1/N2), deep (N3), and REM stages using HRV features. The algorithm uses heart rate level, HRV magnitude (RMSSD), and respiratory rate estimated from the HF peak frequency.

Insight:
The sleep scorer uses HRV-only features rather than EEG, enabling sleep staging from a simple chest strap or wrist sensor. Accuracy is lower than polysomnography but

5 Baseline and Deviation Detection

The Exponentially Weighted Moving Average (EWMA) baseline tracker maintains a running personal baseline for each HRV metric. Deviations beyond configurable thresholds (default: 2 standard deviations) trigger alerts.

```python
class EWMABaseline:
    def __init__(self, alpha: float = 0.1):
        self.alpha = alpha
        self.mean: float | None = None
        self.var: float = 0.0

    def update(self, value: float) -> float | None:
        if self.mean is None:
            self.mean = value
            return None
        self.mean = self.alpha * value + (1 - self.alpha) * self.mean
        self.var = self.alpha * (value - self.mean)**2 + (1 - self.alpha) * self.var
        z_score = (value - self.mean) / max(self.var**0.5, 1e-6)
        return z_score
```

6 Virtual Patient Simulator

The simulator generates synthetic physiological data streams for development and testing without hardware. It models:

- **RR intervals** with configurable mean HR and HRV
- **Circadian rhythm** with sinusoidal HR modulation
- **Activity states** (rest, exercise, sleep) with realistic transitions
- **Anomalies** (tachycardia, bradycardia, arrhythmia) injected at configurable intervals

: Simulator Fidelity

The simulator produces data that passes the same artifact rejection filters as real sensor data. This ensures that analysis code tested against simulated data will work correctly with real hardware without modification.

7 Design Summary

Table 3. Key design decisions.

Decision	Rationale
Task Force HRV standards	Clinical validity; established norms for comparison
EWMA baseline	Adapts to individual physiology; no fixed thresholds
HRV-only sleep scoring	Works with minimal hardware (chest strap or wrist)
Virtual Patient	Hardware-free development and CI testing
NATS messaging	Integration with multi-agent ecosystem; TLS security

References

[1] ProtoCentral, *HealthyPi: Open-source vital signs monitor*, 2024. [Online]. Available: https://healthypi.protocentral.com

[2] Synadia, *NATS: Cloud native messaging system*, 2024. [Online]. Available: https://nats.io

[3] Task Force of the European Society of Cardiology, "Heart rate variability: Standards of measurement, physiological interpretation, and clinical use," *European Heart Journal*, vol. 17, pp. 354–381, 1996.

printlistoffiguresandtables listofdesigndecisions printrevisionhistory printtodolist